

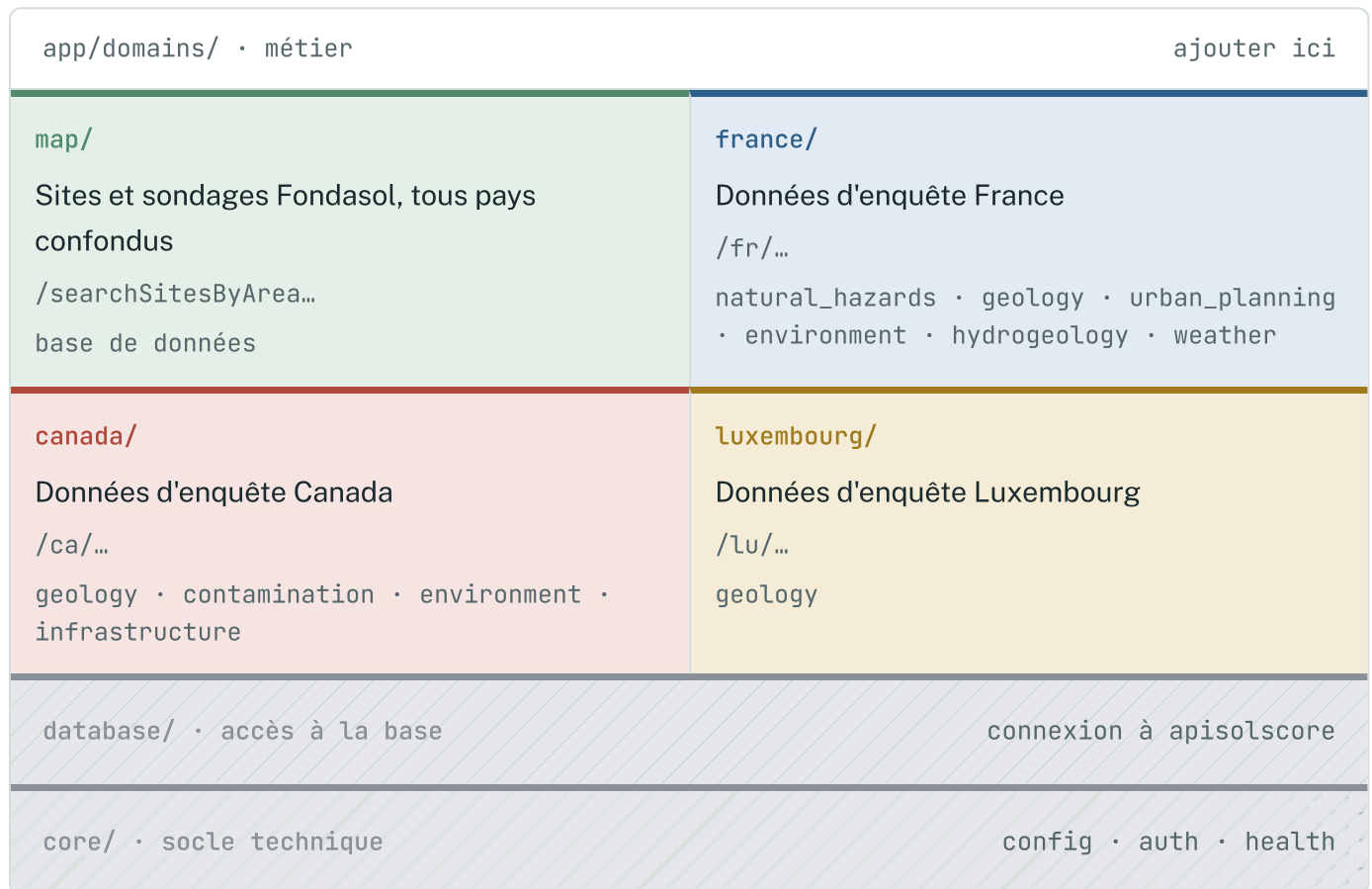
Comment le code est rangé, et pourquoi

Ce projet sert de modèle pour structurer l'API apisolcore. Cette page présente l'organisation des dossiers et les raisons de chaque choix, pour que chacun sache où trouver une fonctionnalité et où en ajouter une nouvelle.

VUE D'ENSEMBLE

Une coupe de l'application

Comme une coupe de sondage, l'application se lit en couches. Tout en bas, le socle technique, partagé par tous. Au-dessus, les domaines métier, indépendants les uns des autres. Une fonctionnalité métier ne touche qu'une couche du haut.



Chaque domaine repose sur `database/` et `core/` ; aucun domaine ne dépend d'un autre.

Les dossiers et leur rôle

back-fastapi/	
├─ app/	
│ └─ main.py	crée l'application et branche les domaines
│ └─ core/	technique, partagé par tous
│ └─ config.py	lit et vérifie le .env
│ └─ auth.py	vérification du jeton SSO Azure
│ └─ health.py	sonde /api/health
│ └─ database/	accès à la base, partagé par tous
│ └─ connection.py	connexion à la base d'apisolscore
│ └─ domains/	métier : c'est ici qu'on ajoute
│ └─ map/	sites et sondages /searchSitesByArea...
│ └─ routes.py	les URL : reçoit la requête, renvoie la réponse
│ └─ schemas.py	le format : vérifie les entrées, décrit les sorties
│ └─ services.py	le métier : appelle les données, applique les règles
│ └─ entities.py	les objets métier : ici, les points de la carte
│ └─ queries.py	le SQL : lit la base de données
│ └─ france/	/fr/...
│ └─ routes.py	regroupe les thèmes
│ └─ natural_hazards/	aléas naturels /fr/natural-hazards/...
│ └─ geology/	
│ └─ urban_planning/	PLU, PPR
│ └─ environment/	sites pollués, CatNat, cavités...
│ └─ hydrogeology/	BSS, ADES
│ └─ weather/	
│ └─ canada/	/ca/...
│ └─ routes.py	regroupe les thèmes
│ └─ geology/	
│ └─ contamination/	GTC, FCSI, sites contaminés...
│ └─ environment/	milieux naturels, faune, eau...
│ └─ infrastructure/	pipelines, RBQ, NPRI
│ └─ luxembourg/	/lu/...
│ └─ routes.py	regroupe les thèmes
│ └─ geology/	
└─ tests/	même arborescence que app/
└─ core/test_auth.py	
└─ domains/map/test_routes.py	
└─ docs/architecture.md	documentation détaillée
└─ Dockerfile · docker-compose.yml	
└─ pyproject.toml	dépendances Python
└─ .env.example	modèle de configuration (le .env n'est pas versionné)

Chaque thème contient déjà tous les fichiers du modèle, vides, et un `routes.py` d'une ligne, déjà branché, avec son préfixe d'URL. Une route ajoutée dans un thème est aussitôt disponible et protégée par le jeton.

LES CHOIX

Pourquoi c'est organisé comme ça

La règle de départ : on découpe là où le code est réellement différent, et on nomme les dossiers avec les mots du métier.

Trois dossiers : core/, database/ et domains/

Où est le technique, où est le métier ?

La racine d' `app/` ne contient que quatre éléments. `core/` regroupe ce qui sert à tout le monde (configuration, sécurité) et `database/` la connexion à la base : on y touche rarement. `domains/` regroupe le métier : c'est là qu'on travaille au quotidien.

Le pays en premier niveau

Pourquoi pas « géologie », puis les pays ?

D'un pays à l'autre, presque rien n'est commun. Les sources diffèrent (Géorisques et le BRGM en France, le Géoportail au Luxembourg, les registres du Québec), les concepts aussi (PPR et ZNIEFF sont français, NPRI et RBQ canadiens), et même la géologie n'a pas la même forme de réponse. Chaque pays évolue à son rythme, sans risquer de casser les autres.

Le thème métier en second niveau

Pourquoi pas un seul dossier par pays ?

La France compte 18 routes dans l'API actuelle, le Canada 24. Un dossier unique par pays deviendrait un fourre-tout. Le thème (aléas naturels, géologie, environnement...) regroupe ce qui change ensemble.

map/ n'est pas découpé par pays

Pourquoi cette exception ?

Ce sont les données internes de Fondasol. Un sondage reste un sondage, qu'il soit en France ou au Canada : le pays n'y est qu'une information parmi d'autres, pas une différence de code.

Les URL suivent les dossiers

Comment retrouver le code d'une route ?

`/fr/natural-hazards/...` est dans `domains/france/natural_hazards/`. Conventions : code pays ISO (`fr`, `ca`, `lu`), noms en anglais, tirets dans les URL, soulignés dans les dossiers. `map/` reste à la racine des URL pour garder le contrat actuel du front.

Les tests reprennent la même arborescence

Où est le test d'un domaine ?

Le test de `app/domains/map/routes.py` est dans `tests/domains/map/test_routes.py`. On le trouve sans chercher, et un domaine se déplace avec ses tests.

À L'INTÉRIEUR D'UN DOMAINE

Toujours les mêmes fichiers

Chaque thème suit la même structure, et tous ces fichiers y sont déjà créés. Pour comprendre une route, on part de `routes.py` et on suit les appels. `map/` est l'exemple complet, sans `sources.py` puisque ses données viennent de la base.

`routes.py`

Les routes : URL, lecture de la requête, appel du service, conversion du résultat en réponse.

Pas de SQL, pas d'appel direct à un service externe, pas de calcul métier.

`schemas.py`

Le contrat de l'API : forme des données reçues et renvoyées, avec leurs contrôles (Pydantic).

Pas d'accès aux données, pas de calcul.

`services.py`

La logique métier et les algorithmes : règles, calculs, croisement de sources.

Pas de notion HTTP (requête, statut, réponse).

`entities.py`

Les objets métier manipulés par les algorithmes (dataclasses) : un site contaminé, un score de risque...

Pas de modèle de table, pas de notion HTTP.

`queries.py`

Le SQL, une fonction par requête, quand le thème lit une base.

Pas de notion HTTP.

`sources.py`

Les appels aux services externes, une fonction par appel.

Pas de mise en forme de la réponse de l'API.

`routes.py` HTTP → `services.py` métier → `sources.py` / `queries.py` données

Toujours dans le même sens

La route appelle le service, le service appelle les données. Aucun fichier n'appelle « vers le haut ». Même quand le service est simple, comme dans `map/`, la route passe par lui : c'est là qu'arrivera la logique métier.

Les algorithmes sont des fonctions pures

Le service récupère d'abord les données, puis appelle les calculs en leur passant ces données. Un calcul ne touche ni la base, ni le réseau, ni HTTP : on le teste avec un petit jeu de données écrit à la main, comme une fonction de notebook.

Un fichier, puis un dossier

Chaque responsabilité tient dans un fichier. Quand il devient trop gros, il devient un dossier du même nom : `services.py` peut donner `algorithms/risk_score.py`, `sources.py` peut donner `sources/gtc.py` et `sources/fcsi.py`.

`schemas.py` ou `entities.py` ?

`schemas.py` décrit ce que voit le front, `entities.py` ce que manipulent les algorithmes. Les services travaillent sur les entités, seules les routes convertissent vers les schémas : on fait évoluer un calcul sans changer la réponse de l'API.

sources.py et queries.py

Ils font le même travail, ramener les données, depuis deux origines : un service externe ou une base. Les deux sont présents ; à la première route, on supprime celui qui ne sert pas. Ils jouent le rôle de repository.

Pas de models.py dans les thèmes

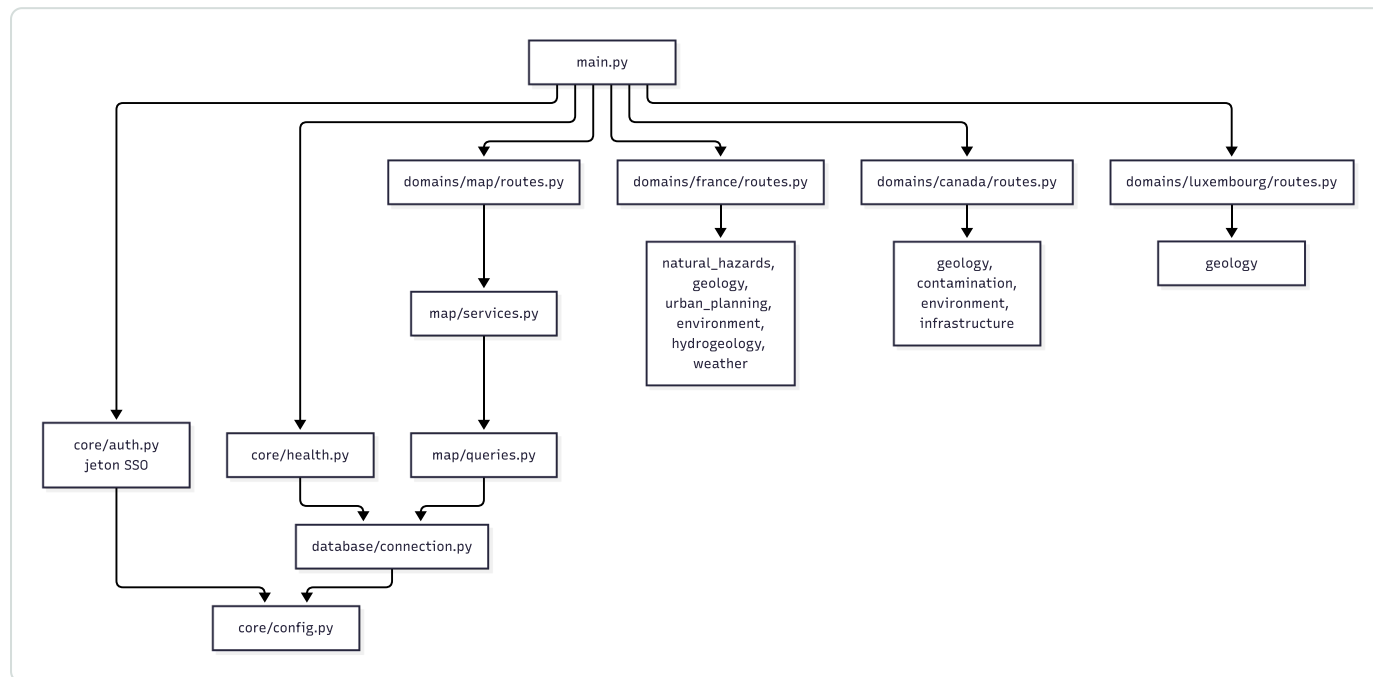
Par convention, `models.py` désigne les modèles SQLAlchemy, qui décrivent les tables.

Une table n'appartient pas à un thème : si on en décrit un jour, ils iront dans

`database/models.py`. Inutile tant que la base est lue en lecture seule.

Qui utilise qui

Un domaine peut utiliser `core/` et `database/` ; eux n'utilisent jamais un domaine, et un pays n'utilise jamais un autre pays. `main.py` connaît les pays, chaque `routes.py` de pays connaît ses thèmes : personne d'autre n'a besoin de la liste complète.



La protection par jeton est posée une seule fois par pays dans `main.py` : toute route ajoutée dans un thème est protégée automatiquement.

Où mettre une nouvelle fonctionnalité ?

LA FONCTIONNALITÉ...	ELLE VA DANS...	EXEMPLE
porte sur les données internes de Fondasol	<code>domains/map/</code> ou un nouveau domaine dans <code>domains/</code>	la fiche d'un sondage : <code>domains/surveys/</code>
repose sur une source propre à un pays	le thème du pays concerné	le zonage sismique : <code>domains/france/natural_hazards/</code>
concerne un sujet nouveau pour un pays	un nouveau thème dans ce pays	<code>domains/france/mining/</code>
concerne un nouveau pays	un nouveau pays dans <code>domains/</code>	<code>domains/belgium/</code> , URL <code>/be/...</code>
sert à tous les domaines	<code>core/</code>	journalisation, gestion des erreurs
concerne la connexion à la base	<code>database/</code>	réglage du pool de connexions

En cas de doute, un petit dossier de plus vaut mieux qu'un dossier fourre-tout.

PAS À PAS

Faire évoluer la structure

Ajouter une route à un thème

1. Écrire l'accès aux données : `sources.py` (service externe) ou `queries.py` (base), et supprimer l'autre.
2. Décrire les objets métier dans `entities.py`.
3. Écrire la logique métier dans `services.py`, les calculs en fonctions pures.
4. Décrire la requête et la réponse dans `schemas.py`.
5. Ajouter la route dans le `routes.py` du thème : elle appelle le service.
6. Écrire ses tests dans `tests/domains/...` : `test_services.py` pour les calculs, `test_routes.py` pour la route.

Ajouter un thème à un pays

1. Copier un thème existant (tous les fichiers du modèle) et changer le préfixe dans `routes.py`.
2. Le brancher dans le `routes.py` du pays.
3. Rien à changer dans `main.py` : le thème hérite du préfixe et de la protection du pays.

Ajouter un pays

1. Créer le dossier du pays dans `domains/`, sur le modèle de `Luxembourg/`.
2. Lui donner son préfixe ISO dans son `routes.py`.
3. Le brancher dans `main.py`, avec la protection par jeton.

Le détail (authentification, configuration, contrat des routes, tests, limites connues) est dans `docs/architecture.md`, à la racine du projet. Le code est écrit en anglais ; la documentation est en français.